



MICROSERVICES COURSE

Session 22

Advanced Patterns: Outbox, Idempotency & API Versioning

Closing Technical Debt from Sessions 4, 7, and 8 · Architecture Clinic #2 Preparation



Dr. ElSayed Mohamed Elsayed Baladoh

Tech Lead · Ph.D. · sayedbaladoh@yahoo.com

DAY

Monday

TIME

3:00–5:30 PM

DURATION

2.5 Hours

PHASE

Phase 3 · Advanced & Enterprise

Where We Are



Session 22 is a Technical Debt Register closeout — closing three items named since Session 8's Architecture Clinic #1 and Session 12's homework.

Three Named Debt Items, Sitting Since Phase 1

"No idempotency on payment retry — duplicate charge risk"

"No API versioning strategy — breaking change risk"

Both named and understood since Phase 1 (Session 8 Architecture Clinic #1 / Session 12 homework) — never fixed. Today we fix them for real.

Plus a third pattern (Outbox) closing a gap you may not have realized existed:

What happens if the Session 7 Saga's database commit succeeds but the Kafka publish that follows it fails?

The Dual-Write Gap Nobody Noticed

Look again at Session 7's `OrderService.createOrder()`:

Step 1: `orderRepository.save(order)` — commits to PostgreSQL

Step 2: `kafkaTemplate.send('order-events', ...)` — publishes to Kafka

TWO SEPARATE systems, TWO SEPARATE network calls, NO shared transaction between them.

If the process crashes between Step 1 succeeding and Step 2 executing:

- The order sits PENDING forever — no exception, no log line
- The customer sees “Order received — processing...” and it never processes
- This has been possible since Session 7, silently, the whole time

Make the Event Part of the SAME Local Transaction

Instead of DB write + Kafka publish as two steps, write to the DB AND to an outbox TABLE in the same PostgreSQL transaction — atomic, already guaranteed by the database (S1's ACID lesson).

ONE Local Transaction

1. `INSERT order`
2. `INSERT outbox_events row`



Separate Outbox Publisher

Polls outbox table → publishes to Kafka → marks row published

If publishing fails, it simply retries later — the row is still sitting safely in the outbox table, not lost.

Outbox Publishing Mechanism

Simple platform, OK with 1-2s polling delay?

→ **Polling Publisher (today's build)**

Need near-zero latency between commit and publish?

→ **Change Data Capture (Debezium)**

Team unfamiliar with CDC tooling, want simplest correct impl?

→ **Polling Publisher**

High-throughput, polling overhead itself becomes bottleneck?

→ **CDC**

Our course: Polling Publisher — simpler, no new infrastructure. Debezium is worth knowing exists but adds real operational surface area for a training platform.

OutboxEvent + Transactional Write (Order Service)

```
@Entity @Table(name = "outbox_events")
public class OutboxEvent {
    @Id @GeneratedValue private Long id;
    private String aggregateType, aggregateId, eventType;
    @Column(columnDefinition = "TEXT") private String payload;
    private boolean published = false;
}

@Transactional // <- both writes, ONE atomic commit, or neither
public OrderResponse createOrder(OrderRequest request) {
    orderRepository.save(order);           // write 1
    outboxRepository.save(outboxEvent);    // write 2 - same tx
    return new OrderResponse(order.getId(), PENDING, "...");
}
```

The Outbox Publisher — A Scheduled Poller

```
@Component
public class OutboxPublisher {
    @Scheduled(fixedDelay = 1000)    // poll every 1 second
    @Transactional
    public void publishPendingEvents() {
        var pending = outboxRepository.findByPublishedFalseOrderByCreatedAtAsc();
        for (OutboxEvent event : pending) {
            kafkaTemplate.send(topicFor(event), event.getAggregateId(), event.getPayload());
            event.setPublished(true);    // marked done — not re-sent
            outboxRepository.save(event);
        }
    }
}

// Enable once: @EnableScheduling on a @Configuration class
```

At-Least-Once, Not Exactly-Once — And Why That's OK



This polling publisher can, in rare failure windows, publish the SAME event twice.

(e.g. crash between `kafkaTemplate.send()` succeeding and `event.setPublished(true)` committing)

What makes at-least-once SAFE in practice:

- Outbox gives you AT-LEAST-ONCE delivery, not exactly-once
- Session 7's Saga consumers already check the current SagaState before acting
- This idempotency guard — built for the Saga — is exactly what makes at-least-once safe

Closing the Session 4 Gap

Recall the exact Session 4 warning (Assessment Bank S04-Q05):

“@Retry(maxAttempts=3) on a method calling Payment Service. Payment succeeds on attempt 1 but the response is lost. Retry attempts 2 and 3 may process the payment again — the customer is charged 2 or 3 times.”

An Idempotency Key breaks the link between “a retry happened” and “the action happened again.”

The CLIENT generates ONE key per logical payment attempt and sends it with every retry of that SAME attempt. The server recognizes repeats and returns the cached result.

Same Key, Same Result — Charge Happens Once

Attempt 1 — key: abc-123

No matching record found → process the REAL charge → store result keyed by abc-123

Attempt 2 (retry) — SAME key: abc-123

Record found for abc-123 → return the CACHED result → no second charge

The key must be born BEFORE the first attempt and survive UNCHANGED across every retry of that same logical operation.

Idempotency Lookup Table + Check-Then-Process

```
@Entity @Table(name = "idempotency_records")
public class IdempotencyRecord {
    @Id private String idempotencyKey;
    @Column(columnDefinition = "TEXT") private String responseBody;
    private int statusCode;
}

@PostMapping
public ResponseEntity<PaymentResponse> processPayment(
    @RequestHeader("Idempotency-Key") String key, @RequestBody PaymentRequest req) {
    var existing = idempotencyRepository.findById(key);
    if (existing.isPresent()) return cachedResponse(existing.get()); // no re-charge
    var response = paymentService.processPayment(req); // REAL charge, once
    idempotencyRepository.save(new IdempotencyRecord(key, response));
    return ResponseEntity.ok(response);
}
```

The Caller Generates ONE Key Outside @Retry

```
// OrderService.java — the CALLER generates and reuses ONE key
String idempotencyKey = UUID.randomUUID().toString();
// generated ONCE, before the @Retry-wrapped call

@CircuitBreaker(name = "paymentService", fallbackMethod = "paymentFallback")
@Retry(name = "paymentService")
public PaymentResponse callPayment(PaymentRequest request, String idempotencyKey) {
    return paymentClient.processPayment(idempotencyKey, request);
    // SAME key on every retry attempt
}
```

Generating the Key INSIDE the Retried Method



If the idempotency key is generated INSIDE the method `@Retry` wraps, every retry attempt gets a DIFFERENT key.

The server can never recognize the retries as duplicates of the same logical request — defeating the entire pattern. You're back to charging the customer multiple times.

The rule: the key must be born once, outside the retried call, and passed IN on every attempt.

Closing the Session 8 Gap

Recall from Session 8's Architecture Clinic Technical Debt Register:

`"No API versioning strategy — /api/products with no version prefix — breaking change risk"`

Why this is real debt:

- Any future breaking change to the response shape has no safe migration path
- Every consumer (Gateway routes, tests, external partners) breaks SIMULTANEOUSLY
- No transition window — fear-driven change avoidance instead of confident evolution

URI-Based vs Header-Based Versioning

`/api/v1/products`

URI-based versioning — CHOSEN

- Simple, visible, cacheable
- Works with the Session 2 Gateway Path predicate
- A new major version = a genuinely new route

`Accept: application/vnd.api.v1+json`

Header-based versioning — not chosen

- More “correct” by some REST purists' standards
- Invisible in logs/browser testing
- Harder to demo live

A stated trade-off, not an objectively “correct” answer over header-based versioning.

/api/v1/ Prefix + Gateway Deprecation Window

```
@RestController @RequestMapping("/api/v1/products") // was: /api/products
public class ProductController { /* unchanged inside */ }

# api-gateway application.yml - two routes
routes:
  - id: product-service-v1
    predicates: [Path=/api/v1/products/**]
  - id: product-service-legacy-redirect # remove after ONE deprecation cycle
    predicates: [Path=/api/products/**]
    filters:
      - RewritePath=/api/products/(?<segment>.*), /api/v1/products/${segment}
      - AddResponseHeader=Deprecation, true
      - AddResponseHeader=Sunset, 'Wed, 01 Oct 2026 00:00:00 GMT'
```

Old clients calling /api/products still work — silently rewritten, with Deprecation/Sunset headers signaling the migration window.

Architecture Clinic #2 — Preparation Starts Today



Session 8 held Architecture Clinic #1 after Phase 1. Session 24 will hold Clinic #2 after the full 24-session journey.

Today's work is the LAST scheduled item on the Session 8/12 Technical Debt Register before that review — so Clinic #2 can ask a genuinely interesting question:

“Of the debt we named in Phase 1, what did we actually fix, and what did we consciously decide to leave, and why?”

Full discussion and Technical Debt Register v2 happen in Session 24 — not today.

Lab 18 — Close One Technical Debt Item End-to-End

Choose ONE of the three patterns and implement it fully — depth over breadth.



Transactional Outbox

OutboxEvent + OutboxPublisher; verify a killed process still publishes on restart



Idempotency Keys

IdempotencyRecord + PaymentController; same key twice → only ONE charge



API Versioning

/api/v1/ prefix + Gateway legacy redirect with Deprecation/Sunset headers

Duration: 10 min in-session + complete as homework · Task D (all trainees): update the Technical Debt Register

Lab 18 — Acceptance Criteria

- ✓ ONE of the three patterns (Outbox / Idempotency / Versioning) is fully implemented and verified
- ✓ The specific failure mode the pattern addresses is demonstrably fixed
- ✓ The Technical Debt Register is updated marking the item RESOLVED with a commit reference
- ✓ A one-sentence prioritization note is written for each of the other two patterns
- ✓ Minimum 2 unit tests pass for the chosen pattern's core logic

```
Commit: session-22: close-[outbox|idempotency|versioning]-technical-debt
```

What We Added Today

 Capability Added

Production-Grade Reliability & API Discipline

The Saga's dual-write gap is closed, payment retries can no longer double-charge, and the platform has a real, dated API versioning strategy.

 Services Updated

- order-service: OutboxEvent + OutboxPublisher
- payment-service: IdempotencyRecord handling
- product-service + api-gateway: /api/v1/ + legacy redirect

Enterprise E-Commerce Platform — After Session 22





Daily Quiz

8 questions · 10 minutes · Google Forms or Kahoot



Session 22 Complete

Reliability & API Discipline — Debt Register Progress

Next: Session 23 — Performance & Load Testing with k6

Wednesday, 3:00–5:30 PM, Online